

Εργασία #2: Μετασχηματισμοί και Προβολές

Βογιατζής Χαρίσιος AEM:9192

10 Μαΐου 2026

github.com/charisvt/cg-auth-hw2-2026

Περίληψη

Σκοπός της εργασίας είναι η υλοποίηση του pipeline προβολής τρισδιάστατων αντικειμένων σε δισδιάστατη εικόνα, καθώς και η παραγωγή δύο animations: ένα με στατική κάμερα και περιστρεφόμενο αντικείμενο, και ένα με κάμερα που τροχιάζει γύρω από ένα στατικό αντικείμενο.

1 Δομή κώδικα και τρόπος κλήσης

Ο κώδικας οργανώνεται στα παρακάτω αρχεία:

- `transformations.py` — κλάση `Trafo` και συναρτήσεις `lookat`, `perspective_project`, `rasterize`, `render_object`.
- `renderer.py` — βοηθητικές συναρτήσεις Gouraud shading από την εργασία #1 (`vector_interp`, `g_shading`).
- `demo1.py` — παράγει 25 frames με στατική κάμερα και περιστρεφόμενο αντικείμενο.
- `demo2.py` — παράγει 25 frames με κινούμενη κάμερα γύρω από το στατικό αντικείμενο.

Εκτέλεση των demos (απαιτείται Python ≥ 3.12 , τα frames αποθηκεύονται στους φακέλους `frames_demo1/` και `frames_demo2/` αντίστοιχα):

```
python demo1.py
python demo2.py
```

2 Κλάση Trafo

Η κλάση `Trafo` υλοποιεί έναν affine μετασχηματισμό (περιστροφή + μετατόπιση) μέσω ενός 3×3 πίνακα περιστροφής `rot_mat` και ενός διανύσματος μετατόπισης `t_vec`.

2.1 translate

Προσθέτει το δοθέν διάνυσμα στο `t_vec`:

```
def translate(self, t_vec: np.ndarray) -> None:
    self.t_vec = self.t_vec + t_vec
```

2.2 rotate

Υπολογίζει τον πίνακα περιστροφής με τον τύπο Rodrigues (σχ. 5.45 σημειώσεων) για περιστροφή γωνίας a γύρω από μοναδιαίο άξονα $\mathbf{u} = [u_x, u_y, u_z]^T$:

$$\mathbf{R} = (1 - \cos a) \begin{bmatrix} u_x^2 & u_x u_y & u_x u_z \\ u_y u_x & u_y^2 & u_y u_z \\ u_z u_x & u_z u_y & u_z^2 \end{bmatrix} + \cos a \mathbf{I}_{3 \times 3} + \sin a \begin{bmatrix} 0 & -u_z & u_y \\ u_z & 0 & -u_x \\ -u_y & u_x & 0 \end{bmatrix}$$

Επειδή ο άξονας διέρχεται από το `center`, το `t_vec` περιστρέφεται ως προς αυτό πριν ενημερωθεί:

```
self.t_vec = np.dot(R_new, self.t_vec - center) + center
self.rot_mat = np.dot(R_new, self.rot_mat)
```

2.3 xform_pnts

Εφαρμόζει τον affine μετασχηματισμό στα σημεία εισόδου ($3 \times N$):

```
def xform_pnts(self, pts: np.ndarray) -> np.ndarray:
    return np.dot(self.rot_mat, pts) + self.t_vec[:, np.newaxis]
```

2.4 sys2sys

Το $\text{sys_src} \in \mathbb{R}^{3 \times 3}$ είναι ο πίνακας περιστροφής που ορίζει το σύστημα συντεταγμένων πηγής (αντίστοιχο της περίπτωσης 2 της §5.4.3 σημειώσεων), ενώ pts είναι τα σημεία εκφρασμένα σε αυτό. Η μετατροπή γίνεται σε δύο βήματα: πηγή $\rightarrow$ WCS, στη συνέχεια $\text{WCS} \rightarrow$ σύστημα προορισμού ($R^{-1} = R^T$ για ορθογώνιο R):

```
pts_world = np.dot(sys_src, pts)
return np.dot(self.rot_mat.T, pts_world - self.t_vec[:, np.newaxis])
```

3 Συνάρτηση lookat

Υπολογίζει τον πίνακα $R \in \mathbb{R}^{3 \times 3}$ και το διάνυσμα $\mathbf{t} \in \mathbb{R}^3$ που μετασχηματίζουν σημεία από το WCS στο σύστημα της κάμερας (§6.3.1–6.3.2 σημειώσεων). Με $C = \text{eye}$, $K = \text{target}$, $\mathbf{u} = \text{up}$:

1. $\hat{z}_c = \frac{\overrightarrow{CK}}{|\overrightarrow{CK}|}$ (άξονας φακού, εξ. 6.6)
2. $\mathbf{t} = \mathbf{u} - \langle \mathbf{u}, \hat{z}_c \rangle \hat{z}_c$, $\hat{y}_c = \frac{\mathbf{t}}{|\mathbf{t}|}$ (διόρθωση up, εξ. 6.7)
3. $\hat{x}_c = \hat{y}_c \times \hat{z}_c$ (εξ. 6.8)
4. $R = \begin{bmatrix} \hat{x}_c^T \\ \hat{y}_c^T \\ \hat{z}_c^T \end{bmatrix}$, $\mathbf{t} = -RC$ (εξ. 6.11, 6.13)

Ο R έχει τα διανύσματα βάσης της κάμερας ως γραμμές (ισοδύναμο με τον R^T των σημειώσεων όπου τα διανύσματα βάσης είναι στήλες). Τα σημεία μπροστά από την κάμερα έχουν $Z_c > 0$.

```
R = np.array([right, up_cam, forward])
t = -np.dot(R, eye)
return R, t
```

4 Συνάρτηση perspective_project

Μετασχηματίζει τα σημεία στο σύστημα της κάμερας και εφαρμόζει την προοπτική προβολή pinhole (§6.2.1, εξ. 6.2 σημειώσεων):

$$p_c = Rp + \mathbf{t}, \quad x_q = w \frac{x_p}{z_p}, \quad y_q = w \frac{y_p}{z_p}$$

όπου w είναι η απόσταση κέντρου προβολής – πετάσματος. Επιστρέφει τις 2D προβολές και το βάθος z_p για την ταξινόμηση κατά βάθος.

```
p_cam = np.dot(R, pts) + t[:, np.newaxis]
depth = p_cam[2]
pts_2d = focal * p_cam[:2] / p_cam[2:3]
return pts_2d, depth
```

5 Συνάρτηση rasterize

Μετατρέπει τις 2D συντεταγμένες από το επίπεδο πετάσματος $([-w/2, w/2] \times [-h/2, h/2])$ σε pixel εικόνας. Ο άξονας της κάμερας αντιστοιχεί στο κέντρο της εικόνας, ενώ ο y αντιστρέφεται επειδή η αρίθμηση ξεκινά από την πάνω αριστερή γωνία:

$$x_{px} = \left(\frac{x}{w} + 0.5\right) w_{res}, \quad y_{px} = \left(0.5 - \frac{y}{h}\right) h_{res}$$

```
x_pix = (pts_2d[0] / plane_w + 0.5) * res_w
y_pix = (0.5 - pts_2d[1] / plane_h) * res_h
return np.round(np.stack([x_pix, y_pix], axis=1)).astype(int)
```

6 Συνάρτηση render_object

Υλοποιεί το πλήρες pipeline απεικόνισης: lookat $\rightarrow$ perspective_project $\rightarrow$ rasterize $\rightarrow$ g_shading. Τα τρίγωνα ταξινομούνται από πίσω προς τα μπρος βάσει του μέσου βάθους των κορυφών τους (Painter's Algorithm), και για κάθε τρίγωνο καλείται η g_shading από την εργασία #1.

```
img = np.ones((res_h, res_w, 3), dtype=np.float64)
avg_depth = np.mean(depth[t_pos_idx], axis=1)
order = np.argsort(avg_depth[::-1])

for idx in order:
    face = t_pos_idx[idx]
    img = g_shading(img, vertices_px[face], v_clr[face])

return img
```

7 Demos

Και τα δύο demos φορτώνουν τις παραμέτρους από το αρχείο data.npy (κάμερα, αντικείμενο, παράμετροι animation). Τα χρώματα των κορυφών (v_clr) προκύπτουν με δειγματοληψία της εικόνας texture loony-repeat.png στις UV συντεταγμένες κάθε κορυφής. Κάθε demo παράγει 25 frames (25 fps $\times$ 1 sec).

7.1 Demo 1 — Στατική κάμερα, περιστρεφόμενο αντικείμενο

Για κάθε frame i δημιουργείται νέο Trafo με γωνία $\theta = 2\pi i/25$ γύρω από τον άξονα Y (κέντρο αρχή αξόνων). Η κάμερα παραμένει σταθερή:

```
trafo = Trafo()
trafo.rotate(np.array([0.0, 1.0, 0.0]), angle, np.array([0.0, 0.0, 0.0]))
v_pos_rot = trafo.xform_pnts(v_pos)
img = render_object(v_pos_rot, v_clr, t_pos_idx, ...)
```

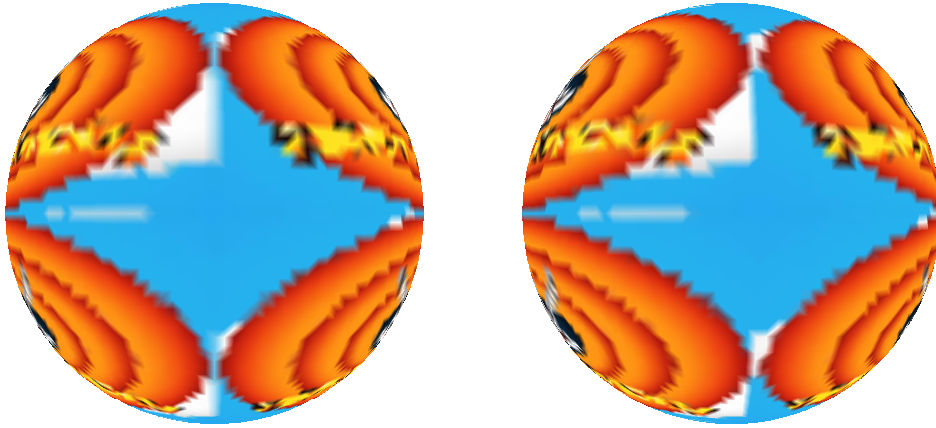
7.2 Demo 2 — Περιστρεφόμενη κάμερα, στατικό αντικείμενο

Η κάμερα κινείται σε κύκλο ακτίνας k_cam_radius στο επίπεδο XZ , κοιτώντας πάντα το αντικείμενο. Η δεξιόστροφη περιστροφή (ως προς $+Y$) εφαρμόζεται με $R_y(-\theta)$:

$$x' = x_0 \cos \theta - z_0 \sin \theta, \quad z' = x_0 \sin \theta + z_0 \cos \theta$$

```
cam_eye = np.array([x0*ct - z0*st, y0, x0*st + z0*ct])
img = render_object(v_pos, v_clr, t_pos_idx, ..., eye=cam_eye, ...)
```

8 Αποτελέσματα



Σχήμα 1: Demo 1 — frames 0 και 6 (περιστροφή αντικειμένου, στατική κάμερα).



Σχήμα 2: Demo 2 — frames 0 και 6 (περιστροφή κάμερας, στατικό αντικείμενο).

9 Δημιουργία video από τα frames

Τα αποθηκευμένα frames μπορούν να συνδυαστούν σε βίντεο με `ffmpeg`:

```
ffmpeg -r 25 -i frames_demo1/frame_%03d.png -vcodec libx264 demo1.mp4  
ffmpeg -r 25 -i frames_demo2/frame_%03d.png -vcodec libx264 demo2.mp4
```

10 Κώδικας

Μπορείτε να βρείτε τον κώδικα και τα αποτελέσματα στο [Github](#).